# Système d'exploitation : TP3 (*threads* et parallélisme)

février 2026 — Pierre Rousselin

## Exercice 1 : Somme et produit

**1.** Télécharger et lire le programme `somme_produit.c` sur la page moodle du cours.

**2.** On va maintenant rendre ce programme multi-thread :

a) un thread est créé pour calculer la somme et un autre pour le produit ;

b) le thread initial attend la fin des deux autres threads et récupère les résultats pour les afficher.

Les types des fonctions `somme` et `produit` doivent changer : la fonction `somme`, par exemple, devient

```
void *somme(void *n);
```

son argument est interprété comme l'adresse d'un entier et la fonction doit retourner l'adresse d'un nouvel entier, alloué sur le tas. (Il sera libéré par le thread initial).

Rappel : pour utiliser les threads POSIX, on doit inclure le fichier d'en-tête `pthread.h` et compiler avec l'option `-pthread`.

Les fonctions utilisées ici sont `pthread_create` et `pthread_join`, voir leur page de manuel (`man pthread_create`) si besoin.

--- * ---

## Correction de l'exercice 1 :

```c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define N_DFT 8

/* Attend l'adresse d'un entier en argument.
 * Calcule la somme 0 + 1 + ... + (*n) et met le résultat dans une variable
 * entière nouvellement allouée. */
void *somme(void *n);

/* Attend l'adresse d'un entier en argument.
 * Calcule le produit 1 * ... * (*n) et met le résultat dans une variable
 * entière nouvellement allouée. */
void *produit(void *n);

int main(int argc, char *argv[])
{
        pthread_t th1, th2;
        int n = argc < 2 ? N_DFT : atoi(argv[1]);
        void *res_somme, *res_prod;

        if (pthread_create(&th1, NULL, somme, &n)) {
                perror("pthread_create somme");
                return 1;
        }
        if (pthread_create(&th2, NULL, produit, &n)) {
                perror("pthread_create produit");
                return 1;
        }
        pthread_join(th1, &res_somme);
        pthread_join(th2, &res_prod);

        printf("n : %d, somme : %d, produit : %d\n", n,
```

```
                        *((int *) res_somme), *((int *) res_prod));
        free(res_somme);
        free(res_prod);
        return 0;
}

void *somme(void *n)
{
        int i, num = *((int *) n);
        int *res = malloc(sizeof(int));

        *res = 0;
        for (i = 1; i <= num; ++i)
                *res += i;

        return res;
}

void *produit(void *n)
{
        int i, num = *((int *) n);
        int *res = malloc(sizeof(int));

        *res = 1;
        for (i = 1; i <= num; ++i)
                *res *= i;

        return res;
}
```

--- * ---

**Exercice 2 : Produit matriciel**

**1.** Télécharger le programme `produit_matrices.c` sur la page moodle du cours et le lire.
Où sont stockées les matrices `A`, `B` et `R` ? Leur taille peut-elle changer au cours du processus ?
Compiler et exécuter le programme produit.

**2.** Les trois lignes de directives pré-processeur

```
#ifndef N /* nombre de lignes des matrices */
#define N 10
#endif
```

signifient : si la macro `N`, n'est pas définie, la définir en lui donnant la valeur `10`.
Entrer les commandes suivantes et dire à quoi sert l'option de compilation `-D`.

```
$ gcc -Wall -DN=3 produit_matrices.c
$ ./a.out
$ gcc -Wall -DN=1 produit_matrices.c
$ ./a.out
```

**3.** Copier le programme pour en écrire une version multi-thread :

```
$ cp produit_matrices.c produit_matrices_N_threads.c
```

Dans cette première version multi-thread il y a `N` threads, c'est-à-dire autant que de lignes :

a) le thread initial crée un thread par ligne, puis attend que les autres aient terminé ;

b) pour chaque $i = 0, ..., N-1$, le thread numéro $i$ calcule les coefficients de la ligne $i$ dans $R$ et retourne `NULL` ;

c) pour savoir sur quelle ligne il travaille, son argument, de type `void *` est interprété comme l'adresse d'une variable entière qui doit contenir $i$. *Attention* au piège classique, si vous donnez l'adresse d'une variable dont le contenu est susceptible de changer, le *thread* pourrait bien travailler avec la mauvaise ligne.

d) Tester le programme avec des valeurs de `N` petites et des affichages pour vérifier qu'il fonctionne, puis commenter les affichages et donner de grandes valeurs pour `N` (typiquement, 1000) pour comparer la durée d'exécution avec la version mono-thread.

**4.** Ça fait beaucoup de `threads`... (donc autant de piles en plus, du boulot pour l'ordonnanceur, etc). Écrire le programme `produit_matrices_K_threads.c` qui contient les directives

```
#ifndef K
#define K 4
#endif
```

Ce programme a `K` threads, peu importe le nombre de lignes. Pour $d = 0, ..., K-1$, le thread numéro $d$ calcule les coefficients des lignes $d, d+K, d+2K, ....$

Tester ce programme :

a) d'abord avec affichages et $N$ petit pour vérifier qu'il fonctionne,

b) puis, sans affichage, avec $N$ de l'ordre de 1000 et différentes valeurs de $K$ (par exemple 1, 2, 4, 8, 16, 128).

c) Voir, avec la commande `lscpu` le nombre de cœurs sur la machine, le nombre de threads par cœurs, le nombre de `CPU`.

--- * ---

**Correction de l'exercice 2 :** Version à $N$ threads :

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
#include <pthread.h>

#ifndef N /* nombre de lignes des matrices */
#define N 10
#endif

void imprime(int matrice[N][N]);
void generation_aleatoire(int matrice[N][N]);
void *calculer_coeffs_prod_ligne(void *declalage);

int A[N][N], B[N][N], R[N][N];

int main()
{
        struct timespec tic, toc;
        double duree;
        pthread_t th[N];
        int i;
        int numeros_thread[N]; /* va contenir { 0, 1, ..., N - 1 } */

        srand(time(NULL));
        generation_aleatoire(A);
        generation_aleatoire(B);
        //imprime(A);
        //puts("");
        //imprime(B);
        //puts("");
```

```
        clock_gettime(CLOCK_REALTIME, &tic);

        for (i = 0; i < N; ++i) {
                numeros_thread[i] = i;
                if (pthread_create(&th[i], NULL, calculer_coeffs_prod_ligne, &numeros_thread[i])) {
                        perror("pthread_create");
                        exit(0);
                }
        }

        for (i = 0; i < N; ++i)
                pthread_join(th[i], NULL);

        clock_gettime(CLOCK_REALTIME, &toc);
        duree = (toc.tv_sec - tic.tv_sec);
        duree += (toc.tv_nsec - tic.tv_nsec) / 1000000000.0;
        printf("durée : %g\n", duree);
        //imprime(R);
        return 0;
}

void imprime(int matrice[N][N])
{
        int i, j;
        for (i = 0; i < N; ++i)
                for (j = 0; j < N; ++j)
                        printf("%d%c", matrice[i][j], j == N - 1 ? '\n' : '\t');
}

void generation_aleatoire(int matrice[N][N])
{
        int i, j;
        for (i = 0; i < N; ++i)
                for (j = 0; j < N; ++j)
                        matrice[i][j] = rand() % 100;
}

void *calculer_coeffs_prod_ligne(void *decalage)
{
        int i = *((int *) decalage), j, k;
        for (j = 0; j < N; ++j) {
                R[i][j] = 0;
                for (k = 0; k < N; ++k)
                        R[i][j] += A[i][k] + B[k][j];
        }
        return NULL;
}
```

Version à $K$ threads :

```
#include <stdlib.h>
#include <stdio.h>
#include <time.h>
#include <pthread.h>

#ifndef N /* nombre de lignes des matrices */
#define N 10
#endif
```

```c
#ifndef K /* nombre de threads */
#define K 4
#endif

void imprime(int matrice[N][N]);
void generation_aleatoire(int matrice[N][N]);
void *calculer_coeffs_prod_ligne(void *declalage);

int A[N][N], B[N][N], R[N][N];

int main()
{
        struct timespec tic, toc;
        double duree;
        pthread_t th[N];
        int i;
        int numeros_thread[K]; /* va contenir { 0, 1, ..., K - 1 } */

        srand(time(NULL));
        generation_aleatoire(A);
        generation_aleatoire(B);
        //imprime(A);
        //puts("");
        //imprime(B);
        //puts("");
        clock_gettime(CLOCK_REALTIME, &tic);

        for (i = 0; i < K; ++i) {
                numeros_thread[i] = i;
                if (pthread_create(&th[i], NULL, calculer_coeffs_prod_ligne, &numeros_thread[i])) {
                        perror("pthread_create");
                        exit(0);
                }
        }

        for (i = 0; i < K; ++i)
                pthread_join(th[i], NULL);

        clock_gettime(CLOCK_REALTIME, &toc);
        duree = (toc.tv_sec - tic.tv_sec);
        duree += (toc.tv_nsec - tic.tv_nsec) / 1000000000.0;
        printf("durée : %g\n", duree);
        //imprime(R);
        return 0;
}

void imprime(int matrice[N][N])
{
        int i, j;
        for (i = 0; i < N; ++i)
                for (j = 0; j < N; ++j)
                        printf("%d%c", matrice[i][j], j == N - 1 ? '\n' : '\t');
}

void generation_aleatoire(int matrice[N][N])
{
```

```
        int i, j;
        for (i = 0; i < N; ++i)
                for (j = 0; j < N; ++j)
                        matrice[i][j] = rand() % 100;
}

void *calculer_coeffs_prod_ligne(void *decalage)
{
        int d = *((int *) decalage), i, j, k;
        for (i = d; i < N; i += K)
                for (j = 0; j < N; ++j) {
                        R[i][j] = 0;
                        for (k = 0; k < N; ++k)
                                R[i][j] += A[i][k] + B[k][j];
                }
        return NULL;
}
```

--- * ---

**Exercice 3 : Méthode des trapèzes**

On va maintenant calculer (un peu naïvement) $\pi$ à l'aide de la méthode des trapèzes. On sait que

$$\pi = 4 \int_0^1 \sqrt{1-x^2}\,\mathrm{d}x$$

parce que la courbe de la fonction à intégrer est un quart de cercle de rayon 1. Pour calculer l'intégrale, on utilise la méthode des trapèzes : l'intervalle $[0,1]$ est découpé en $N$ tout petits intervalles $\left[\frac{i}{N}, \frac{i+1}{N}\right[$ et sur chacun d'eux, l'aire sous la courbe d'équation $y = f(x)$, où $f : x \mapsto \frac{4}{1+x^2}$, est approchée par l'aire du trapèze correspondant :

$$\frac{f\left(\frac{i}{N}\right) + f\left(\frac{i+1}{N}\right)}{2} \times \frac{1}{N}$$

Si vous voulez approfondir, voir, par exemple, `https://fr.wikipedia.org/wiki/M%C3%A9thode_des_trap%C3%A8zes`

Le programme `pi_trapèzes.c`, disponible sur la page moodle du cours, implémente déjà cette méthode.

Remarque : le calcul numérique est parfois une chose fragile. Ici, il faut absolument que $N$ soit une puissance de 2, sinon les erreurs d'arrondi sont importante.

**1.** Télécharger, lire, compiler et exécuter ce programme. On a utilisé la fonction arctan pour comparer la valeur calculée à $4\arctan(1)$ (qui vaut $\pi$) et vérifier le résultat. Il faut donc linker la bibliothèque de mathématiques à l'exécutable produit avec l'option `-lm`.

**2.** On va maintenant donner ces calculs à faire à $K$ threads :

a) le thread principal lance $K$ threads ;

b) le premier de ceux-ci calcule l'aire comprise entre $x = 0$ et $x = \frac{1}{K}$ (non compris) ;

c) le deuxième de ceux-ci calcule l'aire comprise entre $x = \frac{1}{K}$ et $x = \frac{2}{K}$ (non compris) ;

d) ...

e) finalement, le thread initial attend la terminaison des autres, récupère les valeurs calculées et les additionne.

Ainsi, chaque thread (hormis le thread initial) calcule l'aire comprise sous la courbe de la fonction $f$ sur un intervalle $[a, b[$. Pour ce faire, on va utiliser une structure avec 3 champs : l'un pour $a$, l'autre pour $b$ et un dernier champs où il stockera le résultat de son calcul.

Le *thread* initial aura sur sa pile un tableau de $K$ telles structures, qu'il initialisera correctement avant de lancer chaque *thread*.

**3.** Tester le programme avec différentes valeurs de $K$, mais seulement des puissances de 2, sinon on retrouve les problèmes d'arrondis. Il est probable que le résultat soit décevant en terme de performances, le parallélisme étant une petite chose très fragile. Par exemple si les *threads* accèdent trop souvent à « leurs » structures (qui sont dans la pile de `main`), cela les ralentit énormément car celle-ci est « éloignée ».

Il vaut mieux utiliser au maximum des variables *locales* et ne mettre à jour la structure qu'à la fin du calcul. La structure ne devrait être consultée qu'au début du calcul (copier les informations importantes dans des variables locales). En insistant un peu, on peut réussir à (presque) diviser le temps réel d'exécution par le nombre de cœurs de la machine.

--- * ---

**Correction de l'exercice 3 :**

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>

#ifndef K
#define K 4
#endif

#ifndef N
#define N (1 << 27)
#endif
#define PAS (1. / N)

struct calc_pi {
        double depart;
        double fin;
        double res;
};

void *calculer_pi(void *cpi);
double f(double x);

int main(int argc, char *argv[])
{
        pthread_t ths[K];
        struct calc_pi calculs[K];
        int i;
        double pi = 0.;
        struct timespec tic, toc;
        double duree;

        clock_gettime(CLOCK_REALTIME, &tic);

        for (i = 0; i < K; ++i) {
                calculs[i].depart = (double) i / K;
                calculs[i].fin = (double) (i + 1) / K;
                pthread_create(&ths[i], NULL, calculer_pi, &calculs[i]);
        }

        for (i = 0; i < K; ++i) {
                pthread_join(ths[i], NULL);
                pi += calculs[i].res;
```

```
        }

        clock_gettime(CLOCK_REALTIME, &toc);
        duree = (toc.tv_sec - tic.tv_sec);
        duree += (toc.tv_nsec - tic.tv_nsec) / 1000000000.0;

        printf("pi = %.10f\n", pi);
        printf("PI = %.10f\n", atan(1) * 4);
        printf("durée : %g\n", duree);
        return 0;
}

double f(double x)
{
        return sqrt(1 - x * x);
}

void *calculer_pi(void *cpi)
{
        struct calc_pi *c = cpi;
        double x = c->depart;
        double fin = c->fin;
        double res = 0.;
        for (; x < fin; x += PAS)
                res += (f(x) + f(x + PAS)) / 2.;
        res *= 4. * PAS;
        c->res = res;
        return NULL;
}
```

--- * ---